

**15.1-2**

Show, by means of a counterexample, that the following “greedy” strategy does not always determine an optimal way to cut rods. Define the *density* of a rod of length  $i$  to be  $p_i/i$ , that is, its value per inch. The greedy strategy for a rod of length  $n$  cuts off a first piece of length  $i$ , where  $1 \leq i \leq n$ , having maximum density. It then continues by applying the greedy strategy to the remaining piece of length  $n - i$ .

**15.1-3**

Consider a modification of the rod-cutting problem in which, in addition to a price  $p_i$  for each rod, each cut incurs a fixed cost of  $c$ . The revenue associated with a solution is now the sum of the prices of the pieces minus the costs of making the cuts. Give a dynamic-programming algorithm to solve this modified problem.

**15.1-4**

Modify MEMOIZED-CUT-ROD to return not only the value but the actual solution, too.

**15.1-5**

The Fibonacci numbers are defined by recurrence (3.22). Give an  $O(n)$ -time dynamic-programming algorithm to compute the  $n$ th Fibonacci number. Draw the subproblem graph. How many vertices and edges are in the graph?

---

## 15.2 Matrix-chain multiplication

Our next example of dynamic programming is an algorithm that solves the problem of matrix-chain multiplication. We are given a sequence (chain)  $\langle A_1, A_2, \dots, A_n \rangle$  of  $n$  matrices to be multiplied, and we wish to compute the product

$$A_1 A_2 \cdots A_n. \quad (15.5)$$

We can evaluate the expression (15.5) using the standard algorithm for multiplying pairs of matrices as a subroutine once we have parenthesized it to resolve all ambiguities in how the matrices are multiplied together. Matrix multiplication is associative, and so all parenthesizations yield the same product. A product of matrices is *fully parenthesized* if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses. For example, if the chain of matrices is  $\langle A_1, A_2, A_3, A_4 \rangle$ , then we can fully parenthesize the product  $A_1 A_2 A_3 A_4$  in five distinct ways:

$(A_1(A_2(A_3A_4)))$  ,  
 $(A_1((A_2A_3)A_4))$  ,  
 $((A_1A_2)(A_3A_4))$  ,  
 $((A_1(A_2A_3))A_4)$  ,  
 $((A_1A_2)A_3)A_4)$  .

How we parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product. Consider first the cost of multiplying two matrices. The standard algorithm is given by the following pseudocode, which generalizes the SQUARE-MATRIX-MULTIPLY procedure from Section 4.2. The attributes *rows* and *columns* are the numbers of rows and columns in a matrix.

```

MATRIX-MULTIPLY(A, B)
1  if A.columns  $\neq$  B.rows
2      error “incompatible dimensions”
3  else let C be a new A.rows  $\times$  B.columns matrix
4      for i = 1 to A.rows
5          for j = 1 to B.columns
6              cij = 0
7              for k = 1 to A.columns
8                  cij = cij + aik  $\cdot$  bkj
9  return C
  
```

We can multiply two matrices *A* and *B* only if they are **compatible**: the number of columns of *A* must equal the number of rows of *B*. If *A* is a  $p \times q$  matrix and *B* is a  $q \times r$  matrix, the resulting matrix *C* is a  $p \times r$  matrix. The time to compute *C* is dominated by the number of scalar multiplications in line 8, which is  $pqr$ . In what follows, we shall express costs in terms of the number of scalar multiplications.

To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain  $\langle A_1, A_2, A_3 \rangle$  of three matrices. Suppose that the dimensions of the matrices are  $10 \times 100$ ,  $100 \times 5$ , and  $5 \times 50$ , respectively. If we multiply according to the parenthesization  $((A_1A_2)A_3)$ , we perform  $10 \cdot 100 \cdot 5 = 5000$  scalar multiplications to compute the  $10 \times 5$  matrix product  $A_1A_2$ , plus another  $10 \cdot 5 \cdot 50 = 2500$  scalar multiplications to multiply this matrix by  $A_3$ , for a total of 7500 scalar multiplications. If instead we multiply according to the parenthesization  $(A_1(A_2A_3))$ , we perform  $100 \cdot 5 \cdot 50 = 25,000$  scalar multiplications to compute the  $100 \times 50$  matrix product  $A_2A_3$ , plus another  $10 \cdot 100 \cdot 50 = 50,000$  scalar multiplications to multiply  $A_1$  by this matrix, for a total of 75,000 scalar multiplications. Thus, computing the product according to the first parenthesization is 10 times faster.

We state the **matrix-chain multiplication problem** as follows: given a chain  $\langle A_1, A_2, \dots, A_n \rangle$  of  $n$  matrices, where for  $i = 1, 2, \dots, n$ , matrix  $A_i$  has dimension

$p_{i-1} \times p_i$ , fully parenthesize the product  $A_1 A_2 \cdots A_n$  in a way that minimizes the number of scalar multiplications.

Note that in the matrix-chain multiplication problem, we are not actually multiplying matrices. Our goal is only to determine an order for multiplying matrices that has the lowest cost. Typically, the time invested in determining this optimal order is more than paid for by the time saved later on when actually performing the matrix multiplications (such as performing only 7500 scalar multiplications instead of 75,000).

### Counting the number of parenthesizations

Before solving the matrix-chain multiplication problem by dynamic programming, let us convince ourselves that exhaustively checking all possible parenthesizations does not yield an efficient algorithm. Denote the number of alternative parenthesizations of a sequence of  $n$  matrices by  $P(n)$ . When  $n = 1$ , we have just one matrix and therefore only one way to fully parenthesize the matrix product. When  $n \geq 2$ , a fully parenthesized matrix product is the product of two fully parenthesized matrix subproducts, and the split between the two subproducts may occur between the  $k$ th and  $(k + 1)$ st matrices for any  $k = 1, 2, \dots, n - 1$ . Thus, we obtain the recurrence

$$P(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases} \quad (15.6)$$

Problem 12-4 asked you to show that the solution to a similar recurrence is the sequence of **Catalan numbers**, which grows as  $\Omega(4^n/n^{3/2})$ . A simpler exercise (see Exercise 15.2-3) is to show that the solution to the recurrence (15.6) is  $\Omega(2^n)$ . The number of solutions is thus exponential in  $n$ , and the brute-force method of exhaustive search makes for a poor strategy when determining how to optimally parenthesize a matrix chain.

### Applying dynamic programming

We shall use the dynamic-programming method to determine how to optimally parenthesize a matrix chain. In so doing, we shall follow the four-step sequence that we stated at the beginning of this chapter:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution.

4. Construct an optimal solution from computed information.

We shall go through these steps in order, demonstrating clearly how we apply each step to the problem.

**Step 1: The structure of an optimal parenthesization**

For our first step in the dynamic-programming paradigm, we find the optimal substructure and then use it to construct an optimal solution to the problem from optimal solutions to subproblems. In the matrix-chain multiplication problem, we can perform this step as follows. For convenience, let us adopt the notation  $A_{i..j}$ , where  $i \leq j$ , for the matrix that results from evaluating the product  $A_i A_{i+1} \cdots A_j$ . Observe that if the problem is nontrivial, i.e.,  $i < j$ , then to parenthesize the product  $A_i A_{i+1} \cdots A_j$ , we must split the product between  $A_k$  and  $A_{k+1}$  for some integer  $k$  in the range  $i \leq k < j$ . That is, for some value of  $k$ , we first compute the matrices  $A_{i..k}$  and  $A_{k+1..j}$  and then multiply them together to produce the final product  $A_{i..j}$ . The cost of parenthesizing this way is the cost of computing the matrix  $A_{i..k}$ , plus the cost of computing  $A_{k+1..j}$ , plus the cost of multiplying them together.

The optimal substructure of this problem is as follows. Suppose that to optimally parenthesize  $A_i A_{i+1} \cdots A_j$ , we split the product between  $A_k$  and  $A_{k+1}$ . Then the way we parenthesize the “prefix” subchain  $A_i A_{i+1} \cdots A_k$  within this optimal parenthesization of  $A_i A_{i+1} \cdots A_j$  must be an optimal parenthesization of  $A_i A_{i+1} \cdots A_k$ . Why? If there were a less costly way to parenthesize  $A_i A_{i+1} \cdots A_k$ , then we could substitute that parenthesization in the optimal parenthesization of  $A_i A_{i+1} \cdots A_j$  to produce another way to parenthesize  $A_i A_{i+1} \cdots A_j$  whose cost was lower than the optimum: a contradiction. A similar observation holds for how we parenthesize the subchain  $A_{k+1} A_{k+2} \cdots A_j$  in the optimal parenthesization of  $A_i A_{i+1} \cdots A_j$ : it must be an optimal parenthesization of  $A_{k+1} A_{k+2} \cdots A_j$ .

Now we use our optimal substructure to show that we can construct an optimal solution to the problem from optimal solutions to subproblems. We have seen that any solution to a nontrivial instance of the matrix-chain multiplication problem requires us to split the product, and that any optimal solution contains within it optimal solutions to subproblem instances. Thus, we can build an optimal solution to an instance of the matrix-chain multiplication problem by splitting the problem into two subproblems (optimally parenthesizing  $A_i A_{i+1} \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ ), finding optimal solutions to subproblem instances, and then combining these optimal subproblem solutions. We must ensure that when we search for the correct place to split the product, we have considered all possible places, so that we are sure of having examined the optimal one.

### Step 2: A recursive solution

Next, we define the cost of an optimal solution recursively in terms of the optimal solutions to subproblems. For the matrix-chain multiplication problem, we pick as our subproblems the problems of determining the minimum cost of parenthesizing  $A_i A_{i+1} \cdots A_j$  for  $1 \leq i \leq j \leq n$ . Let  $m[i, j]$  be the minimum number of scalar multiplications needed to compute the matrix  $A_{i..j}$ ; for the full problem, the lowest-cost way to compute  $A_{1..n}$  would thus be  $m[1, n]$ .

We can define  $m[i, j]$  recursively as follows. If  $i = j$ , the problem is trivial; the chain consists of just one matrix  $A_{i..i} = A_i$ , so that no scalar multiplications are necessary to compute the product. Thus,  $m[i, i] = 0$  for  $i = 1, 2, \dots, n$ . To compute  $m[i, j]$  when  $i < j$ , we take advantage of the structure of an optimal solution from step 1. Let us assume that to optimally parenthesize, we split the product  $A_i A_{i+1} \cdots A_j$  between  $A_k$  and  $A_{k+1}$ , where  $i \leq k < j$ . Then,  $m[i, j]$  equals the minimum cost for computing the subproducts  $A_{i..k}$  and  $A_{k+1..j}$ , plus the cost of multiplying these two matrices together. Recalling that each matrix  $A_i$  is  $p_{i-1} \times p_i$ , we see that computing the matrix product  $A_{i..k} A_{k+1..j}$  takes  $p_{i-1} p_k p_j$  scalar multiplications. Thus, we obtain

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j .$$

This recursive equation assumes that we know the value of  $k$ , which we do not. There are only  $j - i$  possible values for  $k$ , however, namely  $k = i, i + 1, \dots, j - 1$ . Since the optimal parenthesization must use one of these values for  $k$ , we need only check them all to find the best. Thus, our recursive definition for the minimum cost of parenthesizing the product  $A_i A_{i+1} \cdots A_j$  becomes

$$m[i, j] = \begin{cases} 0 & \text{if } i = j , \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\} & \text{if } i < j . \end{cases} \quad (15.7)$$

The  $m[i, j]$  values give the costs of optimal solutions to subproblems, but they do not provide all the information we need to construct an optimal solution. To help us do so, we define  $s[i, j]$  to be a value of  $k$  at which we split the product  $A_i A_{i+1} \cdots A_j$  in an optimal parenthesization. That is,  $s[i, j]$  equals a value  $k$  such that  $m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$ .

### Step 3: Computing the optimal costs

At this point, we could easily write a recursive algorithm based on recurrence (15.7) to compute the minimum cost  $m[1, n]$  for multiplying  $A_1 A_2 \cdots A_n$ . As we saw for the rod-cutting problem, and as we shall see in Section 15.3, this recursive algorithm takes exponential time, which is no better than the brute-force method of checking each way of parenthesizing the product.

Observe that we have relatively few distinct subproblems: one subproblem for each choice of  $i$  and  $j$  satisfying  $1 \leq i \leq j \leq n$ , or  $\binom{n}{2} + n = \Theta(n^2)$  in all. A recursive algorithm may encounter each subproblem many times in different branches of its recursion tree. This property of overlapping subproblems is the second hallmark of when dynamic programming applies (the first hallmark being optimal substructure).

Instead of computing the solution to recurrence (15.7) recursively, we compute the optimal cost by using a tabular, bottom-up approach. (We present the corresponding top-down approach using memoization in Section 15.3.)

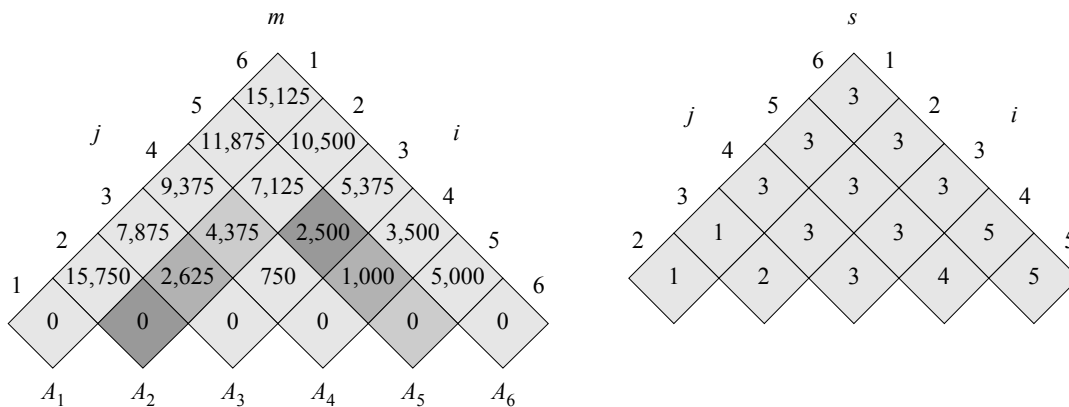
We shall implement the tabular, bottom-up method in the procedure **MATRIX-CHAIN-ORDER**, which appears below. This procedure assumes that matrix  $A_i$  has dimensions  $p_{i-1} \times p_i$  for  $i = 1, 2, \dots, n$ . Its input is a sequence  $p = \langle p_0, p_1, \dots, p_n \rangle$ , where  $p.length = n + 1$ . The procedure uses an auxiliary table  $m[1..n, 1..n]$  for storing the  $m[i, j]$  costs and another auxiliary table  $s[1..n - 1, 2..n]$  that records which index of  $k$  achieved the optimal cost in computing  $m[i, j]$ . We shall use the table  $s$  to construct an optimal solution.

In order to implement the bottom-up approach, we must determine which entries of the table we refer to when computing  $m[i, j]$ . Equation (15.7) shows that the cost  $m[i, j]$  of computing a matrix-chain product of  $j - i + 1$  matrices depends only on the costs of computing matrix-chain products of fewer than  $j - i + 1$  matrices. That is, for  $k = i, i + 1, \dots, j - 1$ , the matrix  $A_{i..k}$  is a product of  $k - i + 1 < j - i + 1$  matrices and the matrix  $A_{k+1..j}$  is a product of  $j - k < j - i + 1$  matrices. Thus, the algorithm should fill in the table  $m$  in a manner that corresponds to solving the parenthesization problem on matrix chains of increasing length. For the subproblem of optimally parenthesizing the chain  $A_i A_{i+1} \cdots A_j$ , we consider the subproblem size to be the length  $j - i + 1$  of the chain.

**MATRIX-CHAIN-ORDER**( $p$ )

```

1   $n = p.length - 1$ 
2  let  $m[1..n, 1..n]$  and  $s[1..n - 1, 2..n]$  be new tables
3  for  $i = 1$  to  $n$ 
4       $m[i, i] = 0$ 
5  for  $l = 2$  to  $n$            //  $l$  is the chain length
6      for  $i = 1$  to  $n - l + 1$ 
7           $j = i + l - 1$ 
8           $m[i, j] = \infty$ 
9          for  $k = i$  to  $j - 1$ 
10              $q = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
11             if  $q < m[i, j]$ 
12                  $m[i, j] = q$ 
13                  $s[i, j] = k$ 
14  return  $m$  and  $s$ 
```



**Figure 15.5** The  $m$  and  $s$  tables computed by MATRIX-CHAIN-ORDER for  $n = 6$  and the following matrix dimensions:

| matrix    | $A_1$          | $A_2$          | $A_3$         | $A_4$         | $A_5$          | $A_6$          |
|-----------|----------------|----------------|---------------|---------------|----------------|----------------|
| dimension | $30 \times 35$ | $35 \times 15$ | $15 \times 5$ | $5 \times 10$ | $10 \times 20$ | $20 \times 25$ |

The tables are rotated so that the main diagonal runs horizontally. The  $m$  table uses only the main diagonal and upper triangle, and the  $s$  table uses only the upper triangle. The minimum number of scalar multiplications to multiply the 6 matrices is  $m[1, 6] = 15,125$ . Of the darker entries, the pairs that have the same shading are taken together in line 10 when computing

$$\begin{aligned}
 m[2, 5] &= \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13,000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11,375 \end{cases} \\
 &= 7125.
 \end{aligned}$$

The algorithm first computes  $m[i, i] = 0$  for  $i = 1, 2, \dots, n$  (the minimum costs for chains of length 1) in lines 3–4. It then uses recurrence (15.7) to compute  $m[i, i + 1]$  for  $i = 1, 2, \dots, n - 1$  (the minimum costs for chains of length  $l = 2$ ) during the first execution of the **for** loop in lines 5–13. The second time through the loop, it computes  $m[i, i + 2]$  for  $i = 1, 2, \dots, n - 2$  (the minimum costs for chains of length  $l = 3$ ), and so forth. At each step, the  $m[i, j]$  cost computed in lines 10–13 depends only on table entries  $m[i, k]$  and  $m[k + 1, j]$  already computed.

Figure 15.5 illustrates this procedure on a chain of  $n = 6$  matrices. Since we have defined  $m[i, j]$  only for  $i \leq j$ , only the portion of the table  $m$  strictly above the main diagonal is used. The figure shows the table rotated to make the main diagonal run horizontally. The matrix chain is listed along the bottom. Using this layout, we can find the minimum cost  $m[i, j]$  for multiplying a subchain  $A_i A_{i+1} \cdots A_j$  of matrices at the intersection of lines running northeast from  $A_i$  and

northwest from  $A_j$ . Each horizontal row in the table contains the entries for matrix chains of the same length. MATRIX-CHAIN-ORDER computes the rows from bottom to top and from left to right within each row. It computes each entry  $m[i, j]$  using the products  $p_{i-1}p_kp_j$  for  $k = i, i + 1, \dots, j - 1$  and all entries southwest and southeast from  $m[i, j]$ .

A simple inspection of the nested loop structure of MATRIX-CHAIN-ORDER yields a running time of  $O(n^3)$  for the algorithm. The loops are nested three deep, and each loop index ( $l$ ,  $i$ , and  $k$ ) takes on at most  $n - 1$  values. Exercise 15.2-5 asks you to show that the running time of this algorithm is in fact also  $\Omega(n^3)$ . The algorithm requires  $\Theta(n^2)$  space to store the  $m$  and  $s$  tables. Thus, MATRIX-CHAIN-ORDER is much more efficient than the exponential-time method of enumerating all possible parenthesizations and checking each one.

#### Step 4: Constructing an optimal solution

Although MATRIX-CHAIN-ORDER determines the optimal number of scalar multiplications needed to compute a matrix-chain product, it does not directly show how to multiply the matrices. The table  $s[1..n - 1, 2..n]$  gives us the information we need to do so. Each entry  $s[i, j]$  records a value of  $k$  such that an optimal parenthesization of  $A_iA_{i+1}\cdots A_j$  splits the product between  $A_k$  and  $A_{k+1}$ . Thus, we know that the final matrix multiplication in computing  $A_{1..n}$  optimally is  $A_{1..s[1,n]}A_{s[1,n]+1..n}$ . We can determine the earlier matrix multiplications recursively, since  $s[1, s[1, n]]$  determines the last matrix multiplication when computing  $A_{1..s[1,n]}$  and  $s[s[1, n] + 1, n]$  determines the last matrix multiplication when computing  $A_{s[1,n]+1..n}$ . The following recursive procedure prints an optimal parenthesization of  $\langle A_i, A_{i+1}, \dots, A_j \rangle$ , given the  $s$  table computed by MATRIX-CHAIN-ORDER and the indices  $i$  and  $j$ . The initial call PRINT-OPTIMAL-PARENS( $s, 1, n$ ) prints an optimal parenthesization of  $\langle A_1, A_2, \dots, A_n \rangle$ .

```

PRINT-OPTIMAL-PARENS( $s, i, j$ )
1  if  $i == j$ 
2      print " $A$ " $i$ 
3  else print "("
4      PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6      print ")"

```

In the example of Figure 15.5, the call PRINT-OPTIMAL-PARENS( $s, 1, 6$ ) prints the parenthesization  $((A_1(A_2A_3))((A_4A_5)A_6))$ .



**Exercises****15.2-1**

Find an optimal parenthesization of a matrix-chain product whose sequence of dimensions is  $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$ .

**15.2-2**

Give a recursive algorithm  $\text{MATRIX-CHAIN-MULTIPLY}(A, s, i, j)$  that actually performs the optimal matrix-chain multiplication, given the sequence of matrices  $\langle A_1, A_2, \dots, A_n \rangle$ , the  $s$  table computed by  $\text{MATRIX-CHAIN-ORDER}$ , and the indices  $i$  and  $j$ . (The initial call would be  $\text{MATRIX-CHAIN-MULTIPLY}(A, s, 1, n)$ .)

**15.2-3**

Use the substitution method to show that the solution to the recurrence (15.6) is  $\Omega(2^n)$ .

**15.2-4**

Describe the subproblem graph for matrix-chain multiplication with an input chain of length  $n$ . How many vertices does it have? How many edges does it have, and which edges are they?

**15.2-5**

Let  $R(i, j)$  be the number of times that table entry  $m[i, j]$  is referenced while computing other table entries in a call of  $\text{MATRIX-CHAIN-ORDER}$ . Show that the total number of references for the entire table is

$$\sum_{i=1}^n \sum_{j=i}^n R(i, j) = \frac{n^3 - n}{3}.$$

(Hint: You may find equation (A.3) useful.)

**15.2-6**

Show that a full parenthesization of an  $n$ -element expression has exactly  $n - 1$  pairs of parentheses.

---

**15.3 Elements of dynamic programming**

Although we have just worked through two examples of the dynamic-programming method, you might still be wondering just when the method applies. From an engineering perspective, when should we look for a dynamic-programming solution to a problem? In this section, we examine the two key ingredients that an opti-

mization problem must have in order for dynamic programming to apply: optimal substructure and overlapping subproblems. We also revisit and discuss more fully how memoization might help us take advantage of the overlapping-subproblems property in a top-down recursive approach.

### Optimal substructure

The first step in solving an optimization problem by dynamic programming is to characterize the structure of an optimal solution. Recall that a problem exhibits **optimal substructure** if an optimal solution to the problem contains within it optimal solutions to subproblems. Whenever a problem exhibits optimal substructure, we have a good clue that dynamic programming might apply. (As Chapter 16 discusses, it also might mean that a greedy strategy applies, however.) In dynamic programming, we build an optimal solution to the problem from optimal solutions to subproblems. Consequently, we must take care to ensure that the range of subproblems we consider includes those used in an optimal solution.

We discovered optimal substructure in both of the problems we have examined in this chapter so far. In Section 15.1, we observed that the optimal way of cutting up a rod of length  $n$  (if we make any cuts at all) involves optimally cutting up the two pieces resulting from the first cut. In Section 15.2, we observed that an optimal parenthesization of  $A_i A_{i+1} \cdots A_j$  that splits the product between  $A_k$  and  $A_{k+1}$  contains within it optimal solutions to the problems of parenthesizing  $A_i A_{i+1} \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ .

You will find yourself following a common pattern in discovering optimal substructure:

1. You show that a solution to the problem consists of making a choice, such as choosing an initial cut in a rod or choosing an index at which to split the matrix chain. Making this choice leaves one or more subproblems to be solved.
2. You suppose that for a given problem, you are given the choice that leads to an optimal solution. You do not concern yourself yet with how to determine this choice. You just assume that it has been given to you.
3. Given this choice, you determine which subproblems ensue and how to best characterize the resulting space of subproblems.
4. You show that the solutions to the subproblems used within an optimal solution to the problem must themselves be optimal by using a “cut-and-paste” technique. You do so by supposing that each of the subproblem solutions is not optimal and then deriving a contradiction. In particular, by “cutting out” the nonoptimal solution to each subproblem and “pasting in” the optimal one, you show that you can get a better solution to the original problem, thus contradicting your supposition that you already had an optimal solution. If an optimal

solution gives rise to more than one subproblem, they are typically so similar that you can modify the cut-and-paste argument for one to apply to the others with little effort.

To characterize the space of subproblems, a good rule of thumb says to try to keep the space as simple as possible and then expand it as necessary. For example, the space of subproblems that we considered for the rod-cutting problem contained the problems of optimally cutting up a rod of length  $i$  for each size  $i$ . This subproblem space worked well, and we had no need to try a more general space of subproblems.

Conversely, suppose that we had tried to constrain our subproblem space for matrix-chain multiplication to matrix products of the form  $A_1 A_2 \cdots A_j$ . As before, an optimal parenthesization must split this product between  $A_k$  and  $A_{k+1}$  for some  $1 \leq k < j$ . Unless we could guarantee that  $k$  always equals  $j - 1$ , we would find that we had subproblems of the form  $A_1 A_2 \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ , and that the latter subproblem is not of the form  $A_1 A_2 \cdots A_j$ . For this problem, we needed to allow our subproblems to vary at “both ends,” that is, to allow both  $i$  and  $j$  to vary in the subproblem  $A_i A_{i+1} \cdots A_j$ .

Optimal substructure varies across problem domains in two ways:

1. how many subproblems an optimal solution to the original problem uses, and
2. how many choices we have in determining which subproblem(s) to use in an optimal solution.

In the rod-cutting problem, an optimal solution for cutting up a rod of size  $n$  uses just one subproblem (of size  $n - i$ ), but we must consider  $n$  choices for  $i$  in order to determine which one yields an optimal solution. Matrix-chain multiplication for the subchain  $A_i A_{i+1} \cdots A_j$  serves as an example with two subproblems and  $j - i$  choices. For a given matrix  $A_k$  at which we split the product, we have two subproblems—parenthesizing  $A_i A_{i+1} \cdots A_k$  and parenthesizing  $A_{k+1} A_{k+2} \cdots A_j$ —and we must solve *both* of them optimally. Once we determine the optimal solutions to subproblems, we choose from among  $j - i$  candidates for the index  $k$ .

Informally, the running time of a dynamic-programming algorithm depends on the product of two factors: the number of subproblems overall and how many choices we look at for each subproblem. In rod cutting, we had  $\Theta(n)$  subproblems overall, and at most  $n$  choices to examine for each, yielding an  $O(n^2)$  running time. Matrix-chain multiplication had  $\Theta(n^2)$  subproblems overall, and in each we had at most  $n - 1$  choices, giving an  $O(n^3)$  running time (actually, a  $\Theta(n^3)$  running time, by Exercise 15.2-5).

Usually, the subproblem graph gives an alternative way to perform the same analysis. Each vertex corresponds to a subproblem, and the choices for a sub-

problem are the edges incident to that subproblem. Recall that in rod cutting, the subproblem graph had  $n$  vertices and at most  $n$  edges per vertex, yielding an  $O(n^2)$  running time. For matrix-chain multiplication, if we were to draw the subproblem graph, it would have  $\Theta(n^2)$  vertices and each vertex would have degree at most  $n - 1$ , giving a total of  $O(n^3)$  vertices and edges.

Dynamic programming often uses optimal substructure in a bottom-up fashion. That is, we first find optimal solutions to subproblems and, having solved the subproblems, we find an optimal solution to the problem. Finding an optimal solution to the problem entails making a choice among subproblems as to which we will use in solving the problem. The cost of the problem solution is usually the subproblem costs plus a cost that is directly attributable to the choice itself. In rod cutting, for example, first we solved the subproblems of determining optimal ways to cut up rods of length  $i$  for  $i = 0, 1, \dots, n - 1$ , and then we determined which such subproblem yielded an optimal solution for a rod of length  $n$ , using equation (15.2). The cost attributable to the choice itself is the term  $p_i$  in equation (15.2). In matrix-chain multiplication, we determined optimal parenthesizations of subchains of  $A_i A_{i+1} \cdots A_j$ , and then we chose the matrix  $A_k$  at which to split the product. The cost attributable to the choice itself is the term  $p_{i-1} p_k p_j$ .

In Chapter 16, we shall examine “greedy algorithms,” which have many similarities to dynamic programming. In particular, problems to which greedy algorithms apply have optimal substructure. One major difference between greedy algorithms and dynamic programming is that instead of first finding optimal solutions to subproblems and then making an informed choice, greedy algorithms first make a “greedy” choice—the choice that looks best at the time—and then solve a resulting subproblem, without bothering to solve all possible related smaller subproblems. Surprisingly, in some cases this strategy works!

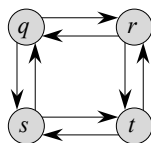
### *Subtleties*

You should be careful not to assume that optimal substructure applies when it does not. Consider the following two problems in which we are given a directed graph  $G = (V, E)$  and vertices  $u, v \in V$ .

**Unweighted shortest path:**<sup>3</sup> Find a path from  $u$  to  $v$  consisting of the fewest edges. Such a path must be simple, since removing a cycle from a path produces a path with fewer edges.

---

<sup>3</sup>We use the term “unweighted” to distinguish this problem from that of finding shortest paths with weighted edges, which we shall see in Chapters 24 and 25. We can use the breadth-first search technique of Chapter 22 to solve the unweighted problem.



**Figure 15.6** A directed graph showing that the problem of finding a longest simple path in an unweighted directed graph does not have optimal substructure. The path  $q \rightarrow r \rightarrow t$  is a longest simple path from  $q$  to  $t$ , but the subpath  $q \rightarrow r$  is not a longest simple path from  $q$  to  $r$ , nor is the subpath  $r \rightarrow t$  a longest simple path from  $r$  to  $t$ .

**Unweighted longest simple path:** Find a simple path from  $u$  to  $v$  consisting of the most edges. We need to include the requirement of simplicity because otherwise we can traverse a cycle as many times as we like to create paths with an arbitrarily large number of edges.

The unweighted shortest-path problem exhibits optimal substructure, as follows. Suppose that  $u \neq v$ , so that the problem is nontrivial. Then, any path  $p$  from  $u$  to  $v$  must contain an intermediate vertex, say  $w$ . (Note that  $w$  may be  $u$  or  $v$ .) Thus, we can decompose the path  $u \xrightarrow{p} v$  into subpaths  $u \xrightarrow{p_1} w \xrightarrow{p_2} v$ . Clearly, the number of edges in  $p$  equals the number of edges in  $p_1$  plus the number of edges in  $p_2$ . We claim that if  $p$  is an optimal (i.e., shortest) path from  $u$  to  $v$ , then  $p_1$  must be a shortest path from  $u$  to  $w$ . Why? We use a “cut-and-paste” argument: if there were another path, say  $p'_1$ , from  $u$  to  $w$  with fewer edges than  $p_1$ , then we could cut out  $p_1$  and paste in  $p'_1$  to produce a path  $u \xrightarrow{p'_1} w \xrightarrow{p_2} v$  with fewer edges than  $p$ , thus contradicting  $p$ ’s optimality. Symmetrically,  $p_2$  must be a shortest path from  $w$  to  $v$ . Thus, we can find a shortest path from  $u$  to  $v$  by considering all intermediate vertices  $w$ , finding a shortest path from  $u$  to  $w$  and a shortest path from  $w$  to  $v$ , and choosing an intermediate vertex  $w$  that yields the overall shortest path. In Section 25.2, we use a variant of this observation of optimal substructure to find a shortest path between every pair of vertices on a weighted, directed graph.

You might be tempted to assume that the problem of finding an unweighted longest simple path exhibits optimal substructure as well. After all, if we decompose a longest simple path  $u \xrightarrow{p} v$  into subpaths  $u \xrightarrow{p_1} w \xrightarrow{p_2} v$ , then mustn’t  $p_1$  be a longest simple path from  $u$  to  $w$ , and mustn’t  $p_2$  be a longest simple path from  $w$  to  $v$ ? The answer is no! Figure 15.6 supplies an example. Consider the path  $q \rightarrow r \rightarrow t$ , which is a longest simple path from  $q$  to  $t$ . Is  $q \rightarrow r$  a longest simple path from  $q$  to  $r$ ? No, for the path  $q \rightarrow s \rightarrow t \rightarrow r$  is a simple path that is longer. Is  $r \rightarrow t$  a longest simple path from  $r$  to  $t$ ? No again, for the path  $r \rightarrow q \rightarrow s \rightarrow t$  is a simple path that is longer.

This example shows that for longest simple paths, not only does the problem lack optimal substructure, but we cannot necessarily assemble a “legal” solution to the problem from solutions to subproblems. If we combine the longest simple paths  $q \rightarrow s \rightarrow t \rightarrow r$  and  $r \rightarrow q \rightarrow s \rightarrow t$ , we get the path  $q \rightarrow s \rightarrow t \rightarrow r \rightarrow q \rightarrow s \rightarrow t$ , which is not simple. Indeed, the problem of finding an unweighted longest simple path does not appear to have any sort of optimal substructure. No efficient dynamic-programming algorithm for this problem has ever been found. In fact, this problem is NP-complete, which—as we shall see in Chapter 34—means that we are unlikely to find a way to solve it in polynomial time.

Why is the substructure of a longest simple path so different from that of a shortest path? Although a solution to a problem for both longest and shortest paths uses two subproblems, the subproblems in finding the longest simple path are not *independent*, whereas for shortest paths they are. What do we mean by subproblems being independent? We mean that the solution to one subproblem does not affect the solution to another subproblem of the same problem. For the example of Figure 15.6, we have the problem of finding a longest simple path from  $q$  to  $t$  with two subproblems: finding longest simple paths from  $q$  to  $r$  and from  $r$  to  $t$ . For the first of these subproblems, we choose the path  $q \rightarrow s \rightarrow t \rightarrow r$ , and so we have also used the vertices  $s$  and  $t$ . We can no longer use these vertices in the second subproblem, since the combination of the two solutions to subproblems would yield a path that is not simple. If we cannot use vertex  $t$  in the second problem, then we cannot solve it at all, since  $t$  is required to be on the path that we find, and it is not the vertex at which we are “splicing” together the subproblem solutions (that vertex being  $r$ ). Because we use vertices  $s$  and  $t$  in one subproblem solution, we cannot use them in the other subproblem solution. We must use at least one of them to solve the other subproblem, however, and we must use both of them to solve it optimally. Thus, we say that these subproblems are not independent. Looked at another way, using resources in solving one subproblem (those resources being vertices) renders them unavailable for the other subproblem.

Why, then, are the subproblems independent for finding a shortest path? The answer is that by nature, the subproblems do not share resources. We claim that if a vertex  $w$  is on a shortest path  $p$  from  $u$  to  $v$ , then we can splice together *any* shortest path  $u \stackrel{p_1}{\rightsquigarrow} w$  and *any* shortest path  $w \stackrel{p_2}{\rightsquigarrow} v$  to produce a shortest path from  $u$  to  $v$ . We are assured that, other than  $w$ , no vertex can appear in both paths  $p_1$  and  $p_2$ . Why? Suppose that some vertex  $x \neq w$  appears in both  $p_1$  and  $p_2$ , so that we can decompose  $p_1$  as  $u \stackrel{p_{ux}}{\rightsquigarrow} x \rightsquigarrow w$  and  $p_2$  as  $w \rightsquigarrow x \stackrel{p_{xv}}{\rightsquigarrow} v$ . By the optimal substructure of this problem, path  $p$  has as many edges as  $p_1$  and  $p_2$  together; let's say that  $p$  has  $e$  edges. Now let us construct a path  $p' = u \stackrel{p_{ux}}{\rightsquigarrow} x \stackrel{p_{xv}}{\rightsquigarrow} v$  from  $u$  to  $v$ . Because we have excised the paths from  $x$  to  $w$  and from  $w$  to  $x$ , each of which contains at least one edge, path  $p'$  contains at most  $e - 2$  edges, which contradicts

the assumption that  $p$  is a shortest path. Thus, we are assured that the subproblems for the shortest-path problem are independent.

Both problems examined in Sections 15.1 and 15.2 have independent subproblems. In matrix-chain multiplication, the subproblems are multiplying subchains  $A_i A_{i+1} \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ . These subchains are disjoint, so that no matrix could possibly be included in both of them. In rod cutting, to determine the best way to cut up a rod of length  $n$ , we look at the best ways of cutting up rods of length  $i$  for  $i = 0, 1, \dots, n - 1$ . Because an optimal solution to the length- $n$  problem includes just one of these subproblem solutions (after we have cut off the first piece), independence of subproblems is not an issue.

### Overlapping subproblems

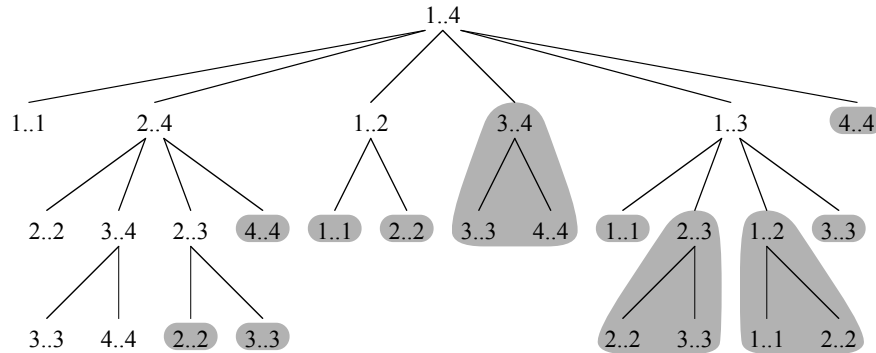
The second ingredient that an optimization problem must have for dynamic programming to apply is that the space of subproblems must be “small” in the sense that a recursive algorithm for the problem solves the same subproblems over and over, rather than always generating new subproblems. Typically, the total number of distinct subproblems is a polynomial in the input size. When a recursive algorithm revisits the same problem repeatedly, we say that the optimization problem has **overlapping subproblems**.<sup>4</sup> In contrast, a problem for which a divide-and-conquer approach is suitable usually generates brand-new problems at each step of the recursion. Dynamic-programming algorithms typically take advantage of overlapping subproblems by solving each subproblem once and then storing the solution in a table where it can be looked up when needed, using constant time per lookup.

In Section 15.1, we briefly examined how a recursive solution to rod cutting makes exponentially many calls to find solutions of smaller subproblems. Our dynamic-programming solution takes an exponential-time recursive algorithm down to quadratic time.

To illustrate the overlapping-subproblems property in greater detail, let us re-examine the matrix-chain multiplication problem. Referring back to Figure 15.5, observe that MATRIX-CHAIN-ORDER repeatedly looks up the solution to subproblems in lower rows when solving subproblems in higher rows. For example, it references entry  $m[3, 4]$  four times: during the computations of  $m[2, 4]$ ,  $m[1, 4]$ ,

---

<sup>4</sup>It may seem strange that dynamic programming relies on subproblems being both independent and overlapping. Although these requirements may sound contradictory, they describe two different notions, rather than two points on the same axis. Two subproblems of the same problem are independent if they do not share resources. Two subproblems are overlapping if they are really the same subproblem that occurs as a subproblem of different problems.



**Figure 15.7** The recursion tree for the computation of `RECURSIVE-MATRIX-CHAIN( $p, 1, 4$ )`. Each node contains the parameters  $i$  and  $j$ . The computations performed in a shaded subtree are replaced by a single table lookup in `MEMOIZED-MATRIX-CHAIN`.

$m[3, 5]$ , and  $m[3, 6]$ . If we were to recompute  $m[3, 4]$  each time, rather than just looking it up, the running time would increase dramatically. To see how, consider the following (inefficient) recursive procedure that determines  $m[i, j]$ , the minimum number of scalar multiplications needed to compute the matrix-chain product  $A_{i..j} = A_i A_{i+1} \cdots A_j$ . The procedure is based directly on the recurrence (15.7).

`RECURSIVE-MATRIX-CHAIN( $p, i, j$ )`

```

1  if  $i == j$ 
2      return 0
3   $m[i, j] = \infty$ 
4  for  $k = i$  to  $j - 1$ 
5       $q = \text{RECURSIVE-MATRIX-CHAIN}(p, i, k)$ 
          +  $\text{RECURSIVE-MATRIX-CHAIN}(p, k + 1, j)$ 
          +  $p_{i-1} p_k p_j$ 
6      if  $q < m[i, j]$ 
7           $m[i, j] = q$ 
8  return  $m[i, j]$ 
```

Figure 15.7 shows the recursion tree produced by the call `RECURSIVE-MATRIX-CHAIN( $p, 1, 4$ )`. Each node is labeled by the values of the parameters  $i$  and  $j$ . Observe that some pairs of values occur many times.

In fact, we can show that the time to compute  $m[1, n]$  by this recursive procedure is at least exponential in  $n$ . Let  $T(n)$  denote the time taken by `RECURSIVE-MATRIX-CHAIN` to compute an optimal parenthesization of a chain of  $n$  matrices. Because the execution of lines 1–2 and of lines 6–7 each take at least unit time, as



does the multiplication in line 5, inspection of the procedure yields the recurrence

$$\begin{aligned} T(1) &\geq 1, \\ T(n) &\geq 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) \quad \text{for } n > 1. \end{aligned}$$

Noting that for  $i = 1, 2, \dots, n-1$ , each term  $T(i)$  appears once as  $T(k)$  and once as  $T(n-k)$ , and collecting the  $n-1$  1s in the summation together with the 1 out front, we can rewrite the recurrence as

$$T(n) \geq 2 \sum_{i=1}^{n-1} T(i) + n. \quad (15.8)$$

We shall prove that  $T(n) = \Omega(2^n)$  using the substitution method. Specifically, we shall show that  $T(n) \geq 2^{n-1}$  for all  $n \geq 1$ . The basis is easy, since  $T(1) \geq 1 = 2^0$ . Inductively, for  $n \geq 2$  we have

$$\begin{aligned} T(n) &\geq 2 \sum_{i=1}^{n-1} 2^{i-1} + n \\ &= 2 \sum_{i=0}^{n-2} 2^i + n \\ &= 2(2^{n-1} - 1) + n \quad (\text{by equation (A.5)}) \\ &= 2^n - 2 + n \\ &\geq 2^{n-1}, \end{aligned}$$

which completes the proof. Thus, the total amount of work performed by the call `RECURSIVE-MATRIX-CHAIN( $p, 1, n$ )` is at least exponential in  $n$ .

Compare this top-down, recursive algorithm (without memoization) with the bottom-up dynamic-programming algorithm. The latter is more efficient because it takes advantage of the overlapping-subproblems property. Matrix-chain multiplication has only  $\Theta(n^2)$  distinct subproblems, and the dynamic-programming algorithm solves each exactly once. The recursive algorithm, on the other hand, must again solve each subproblem every time it reappears in the recursion tree. Whenever a recursion tree for the natural recursive solution to a problem contains the same subproblem repeatedly, and the total number of distinct subproblems is small, dynamic programming can improve efficiency, sometimes dramatically.

### Reconstructing an optimal solution

As a practical matter, we often store which choice we made in each subproblem in a table so that we do not have to reconstruct this information from the costs that we stored.

For matrix-chain multiplication, the table  $s[i, j]$  saves us a significant amount of work when reconstructing an optimal solution. Suppose that we did not maintain the  $s[i, j]$  table, having filled in only the table  $m[i, j]$  containing optimal subproblem costs. We choose from among  $j - i$  possibilities when we determine which subproblems to use in an optimal solution to parenthesizing  $A_i A_{i+1} \cdots A_j$ , and  $j - i$  is not a constant. Therefore, it would take  $\Theta(j - i) = \omega(1)$  time to reconstruct which subproblems we chose for a solution to a given problem. By storing in  $s[i, j]$  the index of the matrix at which we split the product  $A_i A_{i+1} \cdots A_j$ , we can reconstruct each choice in  $O(1)$  time.

### Memoization

As we saw for the rod-cutting problem, there is an alternative approach to dynamic programming that often offers the efficiency of the bottom-up dynamic-programming approach while maintaining a top-down strategy. The idea is to **memoize** the natural, but inefficient, recursive algorithm. As in the bottom-up approach, we maintain a table with subproblem solutions, but the control structure for filling in the table is more like the recursive algorithm.

A memoized recursive algorithm maintains an entry in a table for the solution to each subproblem. Each table entry initially contains a special value to indicate that the entry has yet to be filled in. When the subproblem is first encountered as the recursive algorithm unfolds, its solution is computed and then stored in the table. Each subsequent time that we encounter this subproblem, we simply look up the value stored in the table and return it.<sup>5</sup>

Here is a memoized version of RECURSIVE-MATRIX-CHAIN. Note where it resembles the memoized top-down method for the rod-cutting problem.

---

<sup>5</sup>This approach presupposes that we know the set of all possible subproblem parameters and that we have established the relationship between table positions and subproblems. Another, more general, approach is to memoize by using hashing with the subproblem parameters as keys.

MEMOIZED-MATRIX-CHAIN( $p$ )

```

1   $n = p.length - 1$ 
2  let  $m[1..n, 1..n]$  be a new table
3  for  $i = 1$  to  $n$ 
4      for  $j = i$  to  $n$ 
5           $m[i, j] = \infty$ 
6  return LOOKUP-CHAIN( $m, p, 1, n$ )

```

LOOKUP-CHAIN( $m, p, i, j$ )

```

1  if  $m[i, j] < \infty$ 
2      return  $m[i, j]$ 
3  if  $i == j$ 
4       $m[i, j] = 0$ 
5  else for  $k = i$  to  $j - 1$ 
6       $q = \text{LOOKUP-CHAIN}(m, p, i, k)$ 
        +  $\text{LOOKUP-CHAIN}(m, p, k + 1, j) + p_{i-1}p_kp_j$ 
7      if  $q < m[i, j]$ 
8           $m[i, j] = q$ 
9  return  $m[i, j]$ 

```

The MEMOIZED-MATRIX-CHAIN procedure, like MATRIX-CHAIN-ORDER, maintains a table  $m[1..n, 1..n]$  of computed values of  $m[i, j]$ , the minimum number of scalar multiplications needed to compute the matrix  $A_{i..j}$ . Each table entry initially contains the value  $\infty$  to indicate that the entry has yet to be filled in. Upon calling LOOKUP-CHAIN( $m, p, i, j$ ), if line 1 finds that  $m[i, j] < \infty$ , then the procedure simply returns the previously computed cost  $m[i, j]$  in line 2. Otherwise, the cost is computed as in RECURSIVE-MATRIX-CHAIN, stored in  $m[i, j]$ , and returned. Thus, LOOKUP-CHAIN( $m, p, i, j$ ) always returns the value of  $m[i, j]$ , but it computes it only upon the first call of LOOKUP-CHAIN with these specific values of  $i$  and  $j$ .

Figure 15.7 illustrates how MEMOIZED-MATRIX-CHAIN saves time compared with RECURSIVE-MATRIX-CHAIN. Shaded subtrees represent values that it looks up rather than recomputes.

Like the bottom-up dynamic-programming algorithm MATRIX-CHAIN-ORDER, the procedure MEMOIZED-MATRIX-CHAIN runs in  $O(n^3)$  time. Line 5 of MEMOIZED-MATRIX-CHAIN executes  $\Theta(n^2)$  times. We can categorize the calls of LOOKUP-CHAIN into two types:

1. calls in which  $m[i, j] = \infty$ , so that lines 3–9 execute, and
2. calls in which  $m[i, j] < \infty$ , so that LOOKUP-CHAIN simply returns in line 2.

There are  $\Theta(n^2)$  calls of the first type, one per table entry. All calls of the second type are made as recursive calls by calls of the first type. Whenever a given call of LOOKUP-CHAIN makes recursive calls, it makes  $O(n)$  of them. Therefore, there are  $O(n^3)$  calls of the second type in all. Each call of the second type takes  $O(1)$  time, and each call of the first type takes  $O(n)$  time plus the time spent in its recursive calls. The total time, therefore, is  $O(n^3)$ . Memoization thus turns an  $\Omega(2^n)$ -time algorithm into an  $O(n^3)$ -time algorithm.

In summary, we can solve the matrix-chain multiplication problem by either a top-down, memoized dynamic-programming algorithm or a bottom-up dynamic-programming algorithm in  $O(n^3)$  time. Both methods take advantage of the overlapping-subproblems property. There are only  $\Theta(n^2)$  distinct subproblems in total, and either of these methods computes the solution to each subproblem only once. Without memoization, the natural recursive algorithm runs in exponential time, since solved subproblems are repeatedly solved.

In general practice, if all subproblems must be solved at least once, a bottom-up dynamic-programming algorithm usually outperforms the corresponding top-down memoized algorithm by a constant factor, because the bottom-up algorithm has no overhead for recursion and less overhead for maintaining the table. Moreover, for some problems we can exploit the regular pattern of table accesses in the dynamic-programming algorithm to reduce time or space requirements even further. Alternatively, if some subproblems in the subproblem space need not be solved at all, the memoized solution has the advantage of solving only those subproblems that are definitely required.

## Exercises

### 15.3-1

Which is a more efficient way to determine the optimal number of multiplications in a matrix-chain multiplication problem: enumerating all the ways of parenthesizing the product and computing the number of multiplications for each, or running RECURSIVE-MATRIX-CHAIN? Justify your answer.

### 15.3-2

Draw the recursion tree for the MERGE-SORT procedure from Section 2.3.1 on an array of 16 elements. Explain why memoization fails to speed up a good divide-and-conquer algorithm such as MERGE-SORT.

### 15.3-3

Consider a variant of the matrix-chain multiplication problem in which the goal is to parenthesize the sequence of matrices so as to maximize, rather than minimize,

the number of scalar multiplications. Does this problem exhibit optimal substructure?

### 15.3-4

As stated, in dynamic programming we first solve the subproblems and then choose which of them to use in an optimal solution to the problem. Professor Capulet claims that we do not always need to solve all the subproblems in order to find an optimal solution. She suggests that we can find an optimal solution to the matrix-chain multiplication problem by always choosing the matrix  $A_k$  at which to split the subproduct  $A_i A_{i+1} \cdots A_j$  (by selecting  $k$  to minimize the quantity  $p_{i-1} p_k p_j$ ) *before* solving the subproblems. Find an instance of the matrix-chain multiplication problem for which this greedy approach yields a suboptimal solution.

### 15.3-5

Suppose that in the rod-cutting problem of Section 15.1, we also had limit  $l_i$  on the number of pieces of length  $i$  that we are allowed to produce, for  $i = 1, 2, \dots, n$ . Show that the optimal-substructure property described in Section 15.1 no longer holds.

### 15.3-6

Imagine that you wish to exchange one currency for another. You realize that instead of directly exchanging one currency for another, you might be better off making a series of trades through other currencies, winding up with the currency you want. Suppose that you can trade  $n$  different currencies, numbered  $1, 2, \dots, n$ , where you start with currency 1 and wish to wind up with currency  $n$ . You are given, for each pair of currencies  $i$  and  $j$ , an exchange rate  $r_{ij}$ , meaning that if you start with  $d$  units of currency  $i$ , you can trade for  $dr_{ij}$  units of currency  $j$ . A sequence of trades may entail a commission, which depends on the number of trades you make. Let  $c_k$  be the commission that you are charged when you make  $k$  trades. Show that, if  $c_k = 0$  for all  $k = 1, 2, \dots, n$ , then the problem of finding the best sequence of exchanges from currency 1 to currency  $n$  exhibits optimal substructure. Then show that if commissions  $c_k$  are arbitrary values, then the problem of finding the best sequence of exchanges from currency 1 to currency  $n$  does not necessarily exhibit optimal substructure.

---

## 15.4 Longest common subsequence

Biological applications often need to compare the DNA of two (or more) different organisms. A strand of DNA consists of a string of molecules called

**bases**, where the possible bases are adenine, guanine, cytosine, and thymine. Representing each of these bases by its initial letter, we can express a strand of DNA as a string over the finite set  $\{A, C, G, T\}$ . (See Appendix C for the definition of a string.) For example, the DNA of one organism may be  $S_1 = \text{ACCGGTCGAGTGC GCGGAAGCCGGCCGAA}$ , and the DNA of another organism may be  $S_2 = \text{GTCGTT CGGAATGCCGTTGCTCTGTAAA}$ . One reason to compare two strands of DNA is to determine how “similar” the two strands are, as some measure of how closely related the two organisms are. We can, and do, define similarity in many different ways. For example, we can say that two DNA strands are similar if one is a substring of the other. (Chapter 32 explores algorithms to solve this problem.) In our example, neither  $S_1$  nor  $S_2$  is a substring of the other. Alternatively, we could say that two strands are similar if the number of changes needed to turn one into the other is small. (Problem 15-5 looks at this notion.) Yet another way to measure the similarity of strands  $S_1$  and  $S_2$  is by finding a third strand  $S_3$  in which the bases in  $S_3$  appear in each of  $S_1$  and  $S_2$ ; these bases must appear in the same order, but not necessarily consecutively. The longer the strand  $S_3$  we can find, the more similar  $S_1$  and  $S_2$  are. In our example, the longest strand  $S_3$  is  $\text{GTCGTCGGAAGCCGGCCGAA}$ .

We formalize this last notion of similarity as the longest-common-subsequence problem. A subsequence of a given sequence is just the given sequence with zero or more elements left out. Formally, given a sequence  $X = \langle x_1, x_2, \dots, x_m \rangle$ , another sequence  $Z = \langle z_1, z_2, \dots, z_k \rangle$  is a **subsequence** of  $X$  if there exists a strictly increasing sequence  $\langle i_1, i_2, \dots, i_k \rangle$  of indices of  $X$  such that for all  $j = 1, 2, \dots, k$ , we have  $x_{i_j} = z_j$ . For example,  $Z = \langle B, C, D, B \rangle$  is a subsequence of  $X = \langle A, B, C, B, D, A, B \rangle$  with corresponding index sequence  $\langle 2, 3, 5, 7 \rangle$ .

Given two sequences  $X$  and  $Y$ , we say that a sequence  $Z$  is a **common subsequence** of  $X$  and  $Y$  if  $Z$  is a subsequence of both  $X$  and  $Y$ . For example, if  $X = \langle A, B, C, B, D, A, B \rangle$  and  $Y = \langle B, D, C, A, B, A \rangle$ , the sequence  $\langle B, C, A \rangle$  is a common subsequence of both  $X$  and  $Y$ . The sequence  $\langle B, C, A \rangle$  is not a *longest* common subsequence (LCS) of  $X$  and  $Y$ , however, since it has length 3 and the sequence  $\langle B, C, B, A \rangle$ , which is also common to both  $X$  and  $Y$ , has length 4. The sequence  $\langle B, C, B, A \rangle$  is an LCS of  $X$  and  $Y$ , as is the sequence  $\langle B, D, A, B \rangle$ , since  $X$  and  $Y$  have no common subsequence of length 5 or greater.

In the **longest-common-subsequence problem**, we are given two sequences  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$  and wish to find a maximum-length common subsequence of  $X$  and  $Y$ . This section shows how to efficiently solve the LCS problem using dynamic programming.

### Step 1: Characterizing a longest common subsequence

In a brute-force approach to solving the LCS problem, we would enumerate all subsequences of  $X$  and check each subsequence to see whether it is also a subsequence of  $Y$ , keeping track of the longest subsequence we find. Each subsequence of  $X$  corresponds to a subset of the indices  $\{1, 2, \dots, m\}$  of  $X$ . Because  $X$  has  $2^m$  subsequences, this approach requires exponential time, making it impractical for long sequences.

The LCS problem has an optimal-substructure property, however, as the following theorem shows. As we shall see, the natural classes of subproblems correspond to pairs of “prefixes” of the two input sequences. To be precise, given a sequence  $X = \langle x_1, x_2, \dots, x_m \rangle$ , we define the  $i$ th **prefix** of  $X$ , for  $i = 0, 1, \dots, m$ , as  $X_i = \langle x_1, x_2, \dots, x_i \rangle$ . For example, if  $X = \langle A, B, C, B, D, A, B \rangle$ , then  $X_4 = \langle A, B, C, B \rangle$  and  $X_0$  is the empty sequence.

#### **Theorem 15.1 (Optimal substructure of an LCS)**

Let  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$  be sequences, and let  $Z = \langle z_1, z_2, \dots, z_k \rangle$  be any LCS of  $X$  and  $Y$ .

1. If  $x_m = y_n$ , then  $z_k = x_m = y_n$  and  $Z_{k-1}$  is an LCS of  $X_{m-1}$  and  $Y_{n-1}$ .
2. If  $x_m \neq y_n$ , then  $z_k \neq x_m$  implies that  $Z$  is an LCS of  $X_{m-1}$  and  $Y$ .
3. If  $x_m \neq y_n$ , then  $z_k \neq y_n$  implies that  $Z$  is an LCS of  $X$  and  $Y_{n-1}$ .

**Proof** (1) If  $z_k \neq x_m$ , then we could append  $x_m = y_n$  to  $Z$  to obtain a common subsequence of  $X$  and  $Y$  of length  $k + 1$ , contradicting the supposition that  $Z$  is a *longest* common subsequence of  $X$  and  $Y$ . Thus, we must have  $z_k = x_m = y_n$ . Now, the prefix  $Z_{k-1}$  is a length- $(k - 1)$  common subsequence of  $X_{m-1}$  and  $Y_{n-1}$ . We wish to show that it is an LCS. Suppose for the purpose of contradiction that there exists a common subsequence  $W$  of  $X_{m-1}$  and  $Y_{n-1}$  with length greater than  $k - 1$ . Then, appending  $x_m = y_n$  to  $W$  produces a common subsequence of  $X$  and  $Y$  whose length is greater than  $k$ , which is a contradiction.

(2) If  $z_k \neq x_m$ , then  $Z$  is a common subsequence of  $X_{m-1}$  and  $Y$ . If there were a common subsequence  $W$  of  $X_{m-1}$  and  $Y$  with length greater than  $k$ , then  $W$  would also be a common subsequence of  $X_m$  and  $Y$ , contradicting the assumption that  $Z$  is an LCS of  $X$  and  $Y$ .

(3) The proof is symmetric to (2). ■

The way that Theorem 15.1 characterizes longest common subsequences tells us that an LCS of two sequences contains within it an LCS of prefixes of the two sequences. Thus, the LCS problem has an optimal-substructure property. A recur-

sive solution also has the overlapping-subproblems property, as we shall see in a moment.

### Step 2: A recursive solution

Theorem 15.1 implies that we should examine either one or two subproblems when finding an LCS of  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$ . If  $x_m = y_n$ , we must find an LCS of  $X_{m-1}$  and  $Y_{n-1}$ . Appending  $x_m = y_n$  to this LCS yields an LCS of  $X$  and  $Y$ . If  $x_m \neq y_n$ , then we must solve two subproblems: finding an LCS of  $X_{m-1}$  and  $Y$  and finding an LCS of  $X$  and  $Y_{n-1}$ . Whichever of these two LCSs is longer is an LCS of  $X$  and  $Y$ . Because these cases exhaust all possibilities, we know that one of the optimal subproblem solutions must appear within an LCS of  $X$  and  $Y$ .

We can readily see the overlapping-subproblems property in the LCS problem. To find an LCS of  $X$  and  $Y$ , we may need to find the LCSs of  $X$  and  $Y_{n-1}$  and of  $X_{m-1}$  and  $Y$ . But each of these subproblems has the subsubproblem of finding an LCS of  $X_{m-1}$  and  $Y_{n-1}$ . Many other subproblems share subsubproblems.

As in the matrix-chain multiplication problem, our recursive solution to the LCS problem involves establishing a recurrence for the value of an optimal solution. Let us define  $c[i, j]$  to be the length of an LCS of the sequences  $X_i$  and  $Y_j$ . If either  $i = 0$  or  $j = 0$ , one of the sequences has length 0, and so the LCS has length 0. The optimal substructure of the LCS problem gives the recursive formula

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases} \quad (15.9)$$

Observe that in this recursive formulation, a condition in the problem restricts which subproblems we may consider. When  $x_i = y_j$ , we can and should consider the subproblem of finding an LCS of  $X_{i-1}$  and  $Y_{j-1}$ . Otherwise, we instead consider the two subproblems of finding an LCS of  $X_i$  and  $Y_{j-1}$  and of  $X_{i-1}$  and  $Y_j$ . In the previous dynamic-programming algorithms we have examined—for rod cutting and matrix-chain multiplication—we ruled out no subproblems due to conditions in the problem. Finding an LCS is not the only dynamic-programming algorithm that rules out subproblems based on conditions in the problem. For example, the edit-distance problem (see Problem 15-5) has this characteristic.

### Step 3: Computing the length of an LCS

Based on equation (15.9), we could easily write an exponential-time recursive algorithm to compute the length of an LCS of two sequences. Since the LCS problem



has only  $\Theta(mn)$  distinct subproblems, however, we can use dynamic programming to compute the solutions bottom up.

Procedure **LCS-LENGTH** takes two sequences  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$  as inputs. It stores the  $c[i, j]$  values in a table  $c[0..m, 0..n]$ , and it computes the entries in **row-major** order. (That is, the procedure fills in the first row of  $c$  from left to right, then the second row, and so on.) The procedure also maintains the table  $b[1..m, 1..n]$  to help us construct an optimal solution. Intuitively,  $b[i, j]$  points to the table entry corresponding to the optimal subproblem solution chosen when computing  $c[i, j]$ . The procedure returns the  $b$  and  $c$  tables;  $c[m, n]$  contains the length of an LCS of  $X$  and  $Y$ .

**LCS-LENGTH**( $X, Y$ )

```

1   $m = X.length$ 
2   $n = Y.length$ 
3  let  $b[1..m, 1..n]$  and  $c[0..m, 0..n]$  be new tables
4  for  $i = 1$  to  $m$ 
5       $c[i, 0] = 0$ 
6  for  $j = 0$  to  $n$ 
7       $c[0, j] = 0$ 
8  for  $i = 1$  to  $m$ 
9      for  $j = 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] = c[i - 1, j - 1] + 1$ 
12              $b[i, j] = \nwarrow$ 
13         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] = c[i - 1, j]$ 
15              $b[i, j] = \uparrow$ 
16         else  $c[i, j] = c[i, j - 1]$ 
17              $b[i, j] = \leftarrow$ 
18  return  $c$  and  $b$ 
```

Figure 15.8 shows the tables produced by **LCS-LENGTH** on the sequences  $X = \langle A, B, C, B, D, A, B \rangle$  and  $Y = \langle B, D, C, A, B, A \rangle$ . The running time of the procedure is  $\Theta(mn)$ , since each table entry takes  $\Theta(1)$  time to compute.

#### Step 4: Constructing an LCS

The  $b$  table returned by **LCS-LENGTH** enables us to quickly construct an LCS of  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$ . We simply begin at  $b[m, n]$  and trace through the table by following the arrows. Whenever we encounter a “ $\nwarrow$ ” in entry  $b[i, j]$ , it implies that  $x_i = y_j$  is an element of the LCS that **LCS-LENGTH**

|       |     | $j$   | 0   | 1   | 2   | 3   | 4   | 5   | 6 |
|-------|-----|-------|-----|-----|-----|-----|-----|-----|---|
| $i$   |     | $y_j$ | $B$ | $D$ | $C$ | $A$ | $B$ | $A$ |   |
| $x_i$ |     | 0     | 0   | 0   | 0   | 0   | 0   | 0   |   |
| 1     | $A$ | 0     | ↑   | ↑   | ↑   | ↖   | ←   | ↖   |   |
| 2     | $B$ | 0     | ↖   | 1   | ←   | ←   | ↑   | ↖   | ← |
| 3     | $C$ | 0     | ↑   | ↑   | ↑   | 2   | ←   | 2   | ↑ |
| 4     | $B$ | 0     | ↖   | 1   | ↑   | 2   | 2   | ↖   | ← |
| 5     | $D$ | 0     | ↑   | ↑   | 2   | 2   | ↑   | 3   | ↑ |
| 6     | $A$ | 0     | ↑   | 1   | 2   | 2   | 3   | ↑   | ↖ |
| 7     | $B$ | 0     | ↖   | 1   | 2   | 2   | 3   | ↖   | 4 |

**Figure 15.8** The  $c$  and  $b$  tables computed by LCS-LENGTH on the sequences  $X = \langle A, B, C, B, D, A, B \rangle$  and  $Y = \langle B, D, C, A, B, A \rangle$ . The square in row  $i$  and column  $j$  contains the value of  $c[i, j]$  and the appropriate arrow for the value of  $b[i, j]$ . The entry 4 in  $c[7, 6]$ —the lower right-hand corner of the table—is the length of an LCS  $\langle B, C, B, A \rangle$  of  $X$  and  $Y$ . For  $i, j > 0$ , entry  $c[i, j]$  depends only on whether  $x_i = y_j$  and the values in entries  $c[i - 1, j]$ ,  $c[i, j - 1]$ , and  $c[i - 1, j - 1]$ , which are computed before  $c[i, j]$ . To reconstruct the elements of an LCS, follow the  $b[i, j]$  arrows from the lower right-hand corner; the sequence is shaded. Each “↖” on the shaded sequence corresponds to an entry (highlighted) for which  $x_i = y_j$  is a member of an LCS.

found. With this method, we encounter the elements of this LCS in reverse order. The following recursive procedure prints out an LCS of  $X$  and  $Y$  in the proper, forward order. The initial call is PRINT-LCS( $b, X, X.length, Y.length$ ).

```

PRINT-LCS( $b, X, i, j$ )
1  if  $i == 0$  or  $j == 0$ 
2      return
3  if  $b[i, j] == \text{“}\nwarrow\text{”}$ 
4      PRINT-LCS( $b, X, i - 1, j - 1$ )
5      print  $x_i$ 
6  elseif  $b[i, j] == \text{“}\uparrow\text{”}$ 
7      PRINT-LCS( $b, X, i - 1, j$ )
8  else PRINT-LCS( $b, X, i, j - 1$ )

```

For the  $b$  table in Figure 15.8, this procedure prints  $BCBA$ . The procedure takes time  $O(m + n)$ , since it decrements at least one of  $i$  and  $j$  in each recursive call.

### Improving the code

Once you have developed an algorithm, you will often find that you can improve on the time or space it uses. Some changes can simplify the code and improve constant factors but otherwise yield no asymptotic improvement in performance. Others can yield substantial asymptotic savings in time and space.

In the LCS algorithm, for example, we can eliminate the  $b$  table altogether. Each  $c[i, j]$  entry depends on only three other  $c$  table entries:  $c[i - 1, j - 1]$ ,  $c[i - 1, j]$ , and  $c[i, j - 1]$ . Given the value of  $c[i, j]$ , we can determine in  $O(1)$  time which of these three values was used to compute  $c[i, j]$ , without inspecting table  $b$ . Thus, we can reconstruct an LCS in  $O(m + n)$  time using a procedure similar to PRINT-LCS. (Exercise 15.4-2 asks you to give the pseudocode.) Although we save  $\Theta(mn)$  space by this method, the auxiliary space requirement for computing an LCS does not asymptotically decrease, since we need  $\Theta(mn)$  space for the  $c$  table anyway.

We can, however, reduce the asymptotic space requirements for LCS-LENGTH, since it needs only two rows of table  $c$  at a time: the row being computed and the previous row. (In fact, as Exercise 15.4-4 asks you to show, we can use only slightly more than the space for one row of  $c$  to compute the length of an LCS.) This improvement works if we need only the length of an LCS; if we need to reconstruct the elements of an LCS, the smaller table does not keep enough information to retrace our steps in  $O(m + n)$  time.

### Exercises

#### 15.4-1

Determine an LCS of  $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$  and  $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$ .

#### 15.4-2

Give pseudocode to reconstruct an LCS from the completed  $c$  table and the original sequences  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$  in  $O(m + n)$  time, without using the  $b$  table.

#### 15.4-3

Give a memoized version of LCS-LENGTH that runs in  $O(mn)$  time.

#### 15.4-4

Show how to compute the length of an LCS using only  $2 \cdot \min(m, n)$  entries in the  $c$  table plus  $O(1)$  additional space. Then show how to do the same thing, but using  $\min(m, n)$  entries plus  $O(1)$  additional space.

**15.4-5**

Give an  $O(n^2)$ -time algorithm to find the longest monotonically increasing subsequence of a sequence of  $n$  numbers.

**15.4-6 ★**

Give an  $O(n \lg n)$ -time algorithm to find the longest monotonically increasing subsequence of a sequence of  $n$  numbers. (*Hint:* Observe that the last element of a candidate subsequence of length  $i$  is at least as large as the last element of a candidate subsequence of length  $i - 1$ . Maintain candidate subsequences by linking them through the input sequence.)

---

**15.5 Optimal binary search trees**

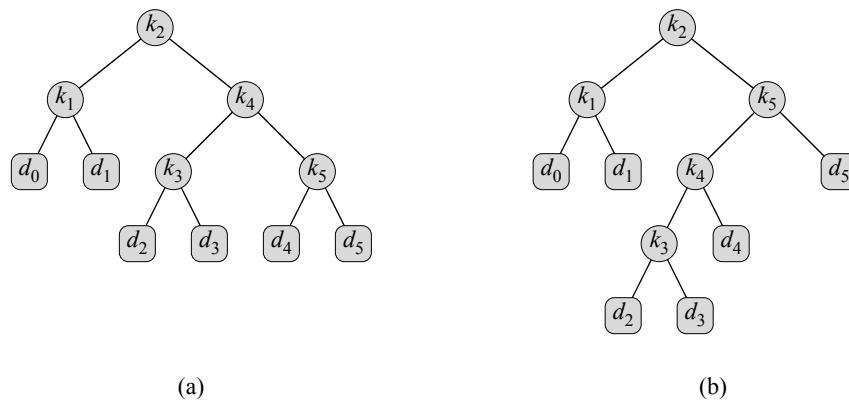
Suppose that we are designing a program to translate text from English to French. For each occurrence of each English word in the text, we need to look up its French equivalent. We could perform these lookup operations by building a binary search tree with  $n$  English words as keys and their French equivalents as satellite data. Because we will search the tree for each individual word in the text, we want the total time spent searching to be as low as possible. We could ensure an  $O(\lg n)$  search time per occurrence by using a red-black tree or any other balanced binary search tree. Words appear with different frequencies, however, and a frequently used word such as *the* may appear far from the root while a rarely used word such as *machicolation* appears near the root. Such an organization would slow down the translation, since the number of nodes visited when searching for a key in a binary search tree equals one plus the depth of the node containing the key. We want words that occur frequently in the text to be placed nearer the root.<sup>6</sup> Moreover, some words in the text might have no French translation,<sup>7</sup> and such words would not appear in the binary search tree at all. How do we organize a binary search tree so as to minimize the number of nodes visited in all searches, given that we know how often each word occurs?

What we need is known as an **optimal binary search tree**. Formally, we are given a sequence  $K = \langle k_1, k_2, \dots, k_n \rangle$  of  $n$  distinct keys in sorted order (so that  $k_1 < k_2 < \dots < k_n$ ), and we wish to build a binary search tree from these keys. For each key  $k_i$ , we have a probability  $p_i$  that a search will be for  $k_i$ . Some searches may be for values not in  $K$ , and so we also have  $n + 1$  “dummy keys”

---

<sup>6</sup>If the subject of the text is castle architecture, we might want *machicolation* to appear near the root.

<sup>7</sup>Yes, *machicolation* has a French counterpart: *mâchicoulis*.



**Figure 15.9** Two binary search trees for a set of  $n = 5$  keys with the following probabilities:

| $i$   | 0    | 1    | 2    | 3    | 4    | 5    |
|-------|------|------|------|------|------|------|
| $p_i$ |      | 0.15 | 0.10 | 0.05 | 0.10 | 0.20 |
| $q_i$ | 0.05 | 0.10 | 0.05 | 0.05 | 0.05 | 0.10 |

**(a)** A binary search tree with expected search cost 2.80. **(b)** A binary search tree with expected search cost 2.75. This tree is optimal.

$d_0, d_1, d_2, \dots, d_n$  representing values not in  $K$ . In particular,  $d_0$  represents all values less than  $k_1$ ,  $d_n$  represents all values greater than  $k_n$ , and for  $i = 1, 2, \dots, n-1$ , the dummy key  $d_i$  represents all values between  $k_i$  and  $k_{i+1}$ . For each dummy key  $d_i$ , we have a probability  $q_i$  that a search will correspond to  $d_i$ . Figure 15.9 shows two binary search trees for a set of  $n = 5$  keys. Each key  $k_i$  is an internal node, and each dummy key  $d_i$  is a leaf. Every search is either successful (finding some key  $k_i$ ) or unsuccessful (finding some dummy key  $d_i$ ), and so we have

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1. \quad (15.10)$$

Because we have probabilities of searches for each key and each dummy key, we can determine the expected cost of a search in a given binary search tree  $T$ . Let us assume that the actual cost of a search equals the number of nodes examined, i.e., the depth of the node found by the search in  $T$ , plus 1. Then the expected cost of a search in  $T$  is

$$\begin{aligned} E[\text{search cost in } T] &= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}_T(d_i) + 1) \cdot q_i \\ &= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i, \end{aligned} \quad (15.11)$$

where  $\text{depth}_T$  denotes a node's depth in the tree  $T$ . The last equality follows from equation (15.10). In Figure 15.9(a), we can calculate the expected search cost node by node:

| node  | depth | probability | contribution |
|-------|-------|-------------|--------------|
| $k_1$ | 1     | 0.15        | 0.30         |
| $k_2$ | 0     | 0.10        | 0.10         |
| $k_3$ | 2     | 0.05        | 0.15         |
| $k_4$ | 1     | 0.10        | 0.20         |
| $k_5$ | 2     | 0.20        | 0.60         |
| $d_0$ | 2     | 0.05        | 0.15         |
| $d_1$ | 2     | 0.10        | 0.30         |
| $d_2$ | 3     | 0.05        | 0.20         |
| $d_3$ | 3     | 0.05        | 0.20         |
| $d_4$ | 3     | 0.05        | 0.20         |
| $d_5$ | 3     | 0.10        | 0.40         |
| Total |       |             | 2.80         |

For a given set of probabilities, we wish to construct a binary search tree whose expected search cost is smallest. We call such a tree an **optimal binary search tree**. Figure 15.9(b) shows an optimal binary search tree for the probabilities given in the figure caption; its expected cost is 2.75. This example shows that an optimal binary search tree is not necessarily a tree whose overall height is smallest. Nor can we necessarily construct an optimal binary search tree by always putting the key with the greatest probability at the root. Here, key  $k_5$  has the greatest search probability of any key, yet the root of the optimal binary search tree shown is  $k_2$ . (The lowest expected cost of any binary search tree with  $k_5$  at the root is 2.85.)

As with matrix-chain multiplication, exhaustive checking of all possibilities fails to yield an efficient algorithm. We can label the nodes of any  $n$ -node binary tree with the keys  $k_1, k_2, \dots, k_n$  to construct a binary search tree, and then add in the dummy keys as leaves. In Problem 12-4, we saw that the number of binary trees with  $n$  nodes is  $\Omega(4^n/n^{3/2})$ , and so we would have to examine an exponential number of binary search trees in an exhaustive search. Not surprisingly, we shall solve this problem with dynamic programming.

### Step 1: The structure of an optimal binary search tree

To characterize the optimal substructure of optimal binary search trees, we start with an observation about subtrees. Consider any subtree of a binary search tree. It must contain keys in a contiguous range  $k_i, \dots, k_j$ , for some  $1 \leq i \leq j \leq n$ . In addition, a subtree that contains keys  $k_i, \dots, k_j$  must also have as its leaves the dummy keys  $d_{i-1}, \dots, d_j$ .

Now we can state the optimal substructure: if an optimal binary search tree  $T$  has a subtree  $T'$  containing keys  $k_i, \dots, k_j$ , then this subtree  $T'$  must be optimal as